

Standard Template Library: STL

- Extends the core C++ language
- Provides a set of common classes and interfaces
- Gives programmers a higher level of abstraction for various data storage and manipulation tasks
- Being template based, it improves performance

STL Components

1. Containers

- used to manage collections of objects of a certain kind
- Every kind of container has its own advantages and disadvantages
- Select a container that suits your requirement best

2. Iterators

- used to step through the elements of collections of objects; which may be containers or subsets of containers
- Offer a small and common interface for all container types
- Interface is independent of the internal structure of the collection

Containers

- Two types
 1. Sequence containers
 2. Associative containers

Sequence Containers

- These are ordered collections
- Every element has a certain position
- Position of element is independent of value of element, it depends on time and place of insertion

Example

1. Vector
2. List
3. Deque

Associative Containers

- These are sorted collections
- Actual position of an element depends on its value as well as sorting criterion
- Ignores the order of insertion

Example

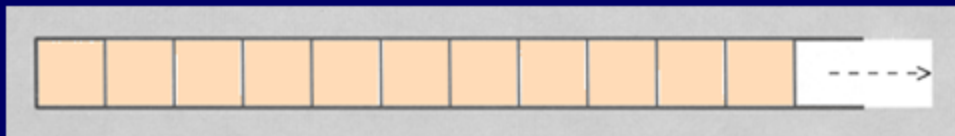
1. Set
2. Multiset
3. Map
4. multimap

Note that...

- Use sequence containers where order of elements does matter
- Sequence containers can be sorted using algorithms
- Sorted collections improve search of an element
- Associative containers are automatically sorted at any point of time, No extra effort should be taken, improves performance

Vector

- #include <vector>



- Manages its elements with a dynamic array
- Ordered collection of elements; Index based ordering

```
#include<vector>
using namespace std;

void main()
{
    vector<int> vec_int;
}
```

Vector

- Grows dynamically
- Reallocates new block if needed
- Reallocation results in
 - Drop in performance
 - Invalidation of all references to vector
- Solution to reallocation
 - Create to Capacity
 - Reserve

```
#include<vector>
using namespace std;

void main()
{
    // Empty vector
    vector<int> vec_int;

    // n elements, default initialization
    vector<int> vec_int(10);

    // n elements, initialised to 14
    vector<int> vec_int(10,14);
    ...
    ...
    // reserve memory for 100 elements
    vec_int.reserve(100);
}
```


Vector

- Insertion and deletion at the end is fast, but slow for other position
- Provides random access, provided index is known
- Random access means every element can be accessed in constant time
- Gives Random access Iterator

```
vector<int> vec_int;  
  
for(int i=0;i<10;i++)  
    vec_int.push_back(i+1);  
  
vector<int>::iterator start,end,temp;  
  
start=vec_int.begin();  
  
end=vec_int.end();  
  
for(temp=start ; temp!=end ; temp++)  
    cout<<*temp;
```

Vector: Important functions

- For Non-modifying Operations

Operation	Effect
c.size()	Returns the actual number of elements
c.empty()	Returns whether the container is empty (equivalent to size()==0, but might be faster)
c.max_size()	Returns the maximum number of elements possible
capacity()	Returns the maximum possible number of elements without reallocation
reserve()	Enlarges capacity, if not enough

- For Insert and Remove Operations

c.insert(pos,elem)	Inserts at iterator position pos a copy of elem and returns the position of the new element
c.push_back(elem)	Appends a copy of elem at the end
c.pop_back()	Removes the last element (does not return it)
c.erase(pos)	Removes the element at iterator position pos and returns the position of the next element
c.clear()	Removes all elements (makes the container empty)

Vector: Important functions

- For Iteration

Operation	Effect
c.begin()	Returns a random access iterator for the first element
c.end()	Returns a random access iterator for the position after the last element
c.rbegin()	Returns a reverse iterator for the first element of a reverse iteration
c.rend()	Returns a reverse iterator for the position after the last element of a reverse iteration

- For Direct Element Access

Operation	Effect
c.at(idx)	Returns the element with index idx (throws range error exception if idx is out of range)
c[idx]	Returns the element with index idx (<i>no</i> range checking)
c.front()	Returns the first element
c.back()	Returns the last element

Deque

- `# include <deque>`



- Manages its elements with a dynamic array open at both ends
- Ordered collection of elements; Index based ordering

Deque

```
#include<deque>
using namespace std;

deque<int> dec_int;

for(int i=0;i<10;i++)
{
    if(i%2)
        dec_int.push_back(i+1);
    else
        dec_int.push_front(i+1);
}

deque<int>::iterator start,end,temp;

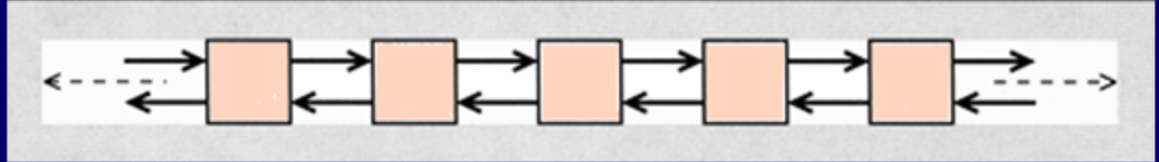
start=dec_int.begin();

end=dec_int.end();

for(temp=start;temp!=end;temp++)
    cout<<*temp;
```

Lists

- `# include <list>`



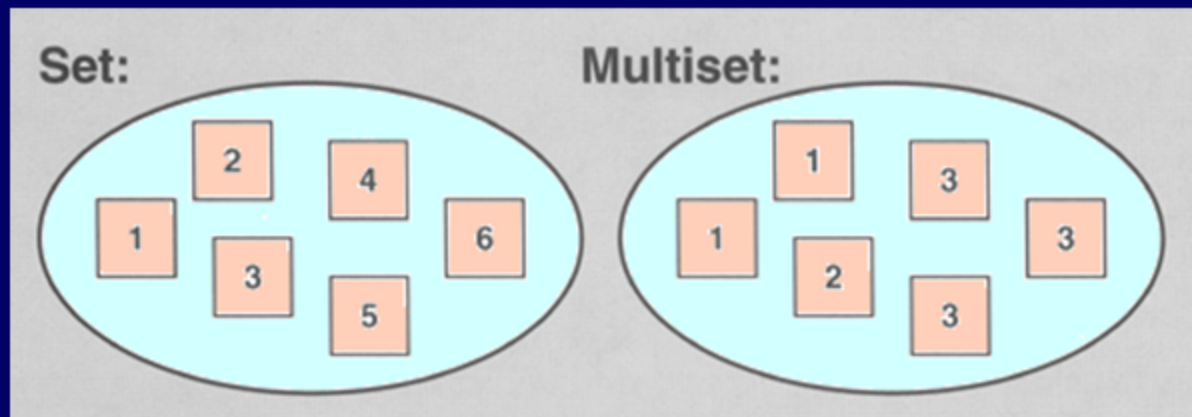
- Manages its elements with a doubly linked list
- Ordered collection of elements; No indexing of elements

Lists

- Does not provide random access, provide sequential iterator
- Hence does not support operations like `at()` , `[ ]` etc
- Element access is not direct, Must navigate from beginning, hence time consuming
- Addition of elements is very fast at any position
- No re-allocation hence no drawbacks of copying and iterator invalidation
- Hence does not support functions like `capacity()`, `reserve()` etc

Set and Multisets

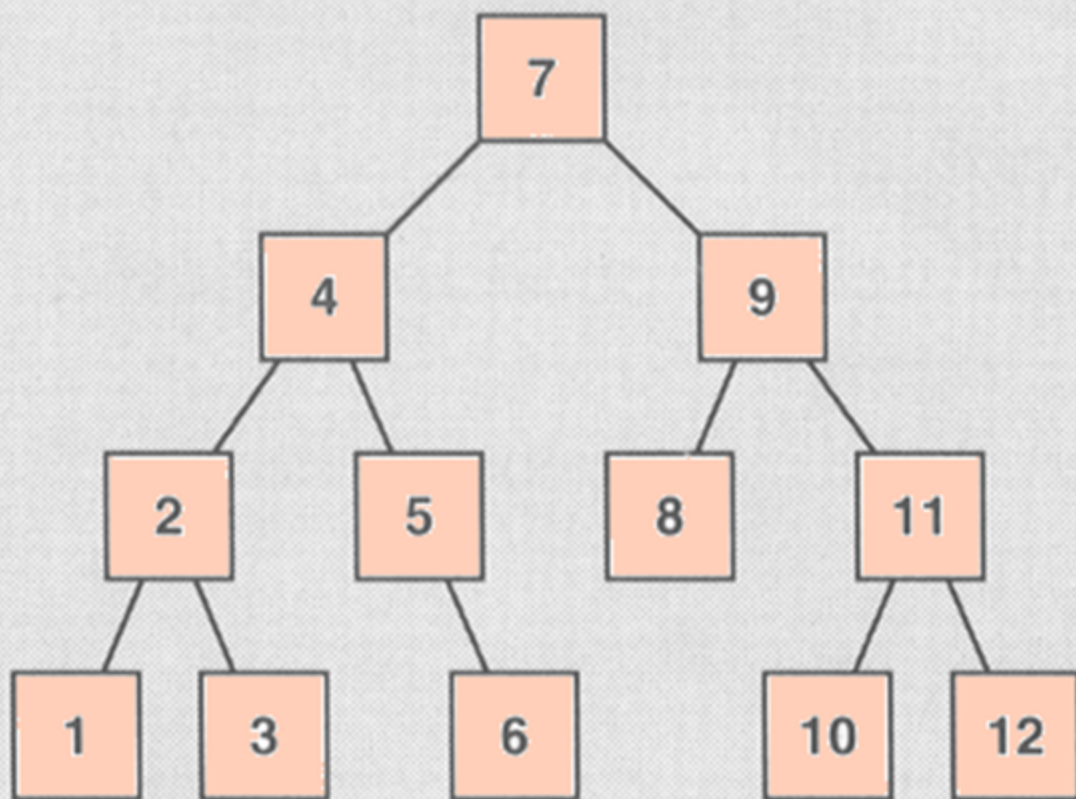
- `# include <set>`



- Sort their elements automatically according to a certain sorting criterion
- Multisets allow duplicates, whereas sets do not

Set and Multisets

- Implemented using tree structure

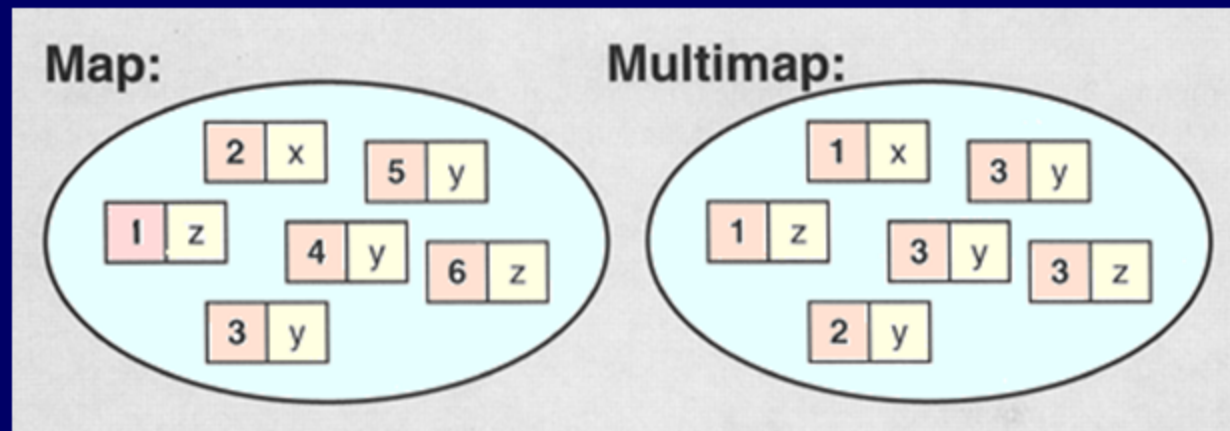


Set and Multisets

- Major Advantage: Binary tree performs well when elements with a certain value are searched
- Major Drawback: Can not change the value of an element directly because this might disturb the correct sorted order. To modify an element, first you have to remove it and again add the new value
- Do not maintain the order of insertion of elements

Maps and Multimaps

- `# include <map>`



- Maintains the data in the form of a key-value pair
- Standardizes associative container
- Maps don't allow duplicate keys, Multimaps allow duplicates
- Improves search performance because key search is faster than the value search
- Similar interface as set and multiset

Maps and Multimaps

```
#include<map>
using namespace std;

void main()
{
    map<int,string> m;

    pair<int,string> p;

    map<int,string>::iterator Start,End,temp;

    p.first=10;
    p.second="India";

    m.insert(p);

    Start= m.begin();
    End= m.end();

    cout<<(*Start).first<<"\t";
    cout<<(*Start).second<<endl;
}
```